Build Task — Reusable Chain Module — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Copy these in unchanged first: `config.py`, `exceptions.py`, `logging_setup.py` from Doc01's Build Task, and `chat_client.py`, `schemas.py` from Doc04's Build Task.

- **`prompts.py`** — `get_extraction_prompt() -> ChatPromptTemplate`, a function, so `chain.py` and the tests get the same prompt.
- **`chain.py`** — `MODEL_NAME = "gpt-4o-mini"` at the top, and `build_extraction_chain(model_name=MODEL_NAME, temperature=0.0) -> Runnable`. Doc01's `Config` holds only the key and the log level, so the model choice lives here, in one named place.
- **`compare_with_raw_sdk.py`** — the test file: `load_config()` first, then one function per Test Case, each printing one `case N` line.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
# chain.py
MODEL_NAME = "gpt-4o-mini"
```

```
def build_extraction_chain(
    model_name: str = MODEL_NAME, temperature: float = 0.0
) -> Runnable:
    logger = get_logger(__name__)
    logger.info(f"Building the extraction chain with {model_name}")
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=model_name, temperature=temperature)
    return prompt | model.with_structured_output(ExtractedInfo)
```

Add the imports, write `prompts.py`, then copy `run_comparison()` from the Real-world exercise into `compare_with_raw_sdk.py` — change only where the chain comes from. Add the two other test functions, then compare against the Solution.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `prompts.py`: the wording is a separate concern from how the chain is built. It reuses Doc04's `EXTRACT_PROMPT`, so the LCEL and raw versions get the same words. **If not:** the two versions would get different prompts, and a mismatch would prove nothing.

```
# prompts.py
from langchain_core.prompts import ChatPromptTemplate

from chat_client import EXTRACT_PROMPT

def get_extraction_prompt() -> ChatPromptTemplate:
    # why: the same system rules Doc04's raw call sends, word for word —
    # the comparison is only fair if both versions get the same prompt
    return ChatPromptTemplate.from_messages(
        [("system", EXTRACT_PROMPT), ("human", "{text}")]
    )
```

Story — `chain.py`: the one module later documents import instead of writing raw SDK calls again. The model name is a named constant with a default parameter, the same shape as the Basic exercise's `build_chain()`. **If not:** every later file would build its own `ChatOpenAI(...)`, and a model swap would mean a search through all of them.

```
# chain.py
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI
```

```
from logging_setup import get_logger
from prompts import get_extraction_prompt
from schemas import ExtractedInfo

# why: the model choice lives in one named place — a model swap is a
# one-line change here, not a search through every file
MODEL_NAME = "gpt-4o-mini"

def build_extraction_chain(
    model_name: str = MODEL_NAME, temperature: float = 0.0
) -> Runnable:
    # how: the same default-parameter shape as the Basic exercise's
    # build_chain() — callers can pass nothing, or pick another model
    logger = get_logger(__name__)
    logger.info(f"Building the extraction chain with {model_name}")

    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=model_name, temperature=temperature)
    return prompt | model.with_structured_output(ExtractedInfo)
```

Story — `compare_with_raw_sdk.py`: the Build Task's test file — one function per Test Case, in the plain `case N` print style of Doc04's `test_chat_client.py`. `run_comparison()` is the Real-world exercise's, unchanged except for where the chain comes from. **If not:** a prompt change could quietly make the two versions disagree, and nothing would tell you.

```
# compare_with_raw_sdk.py
# Runs the 3 Test Cases from the README. Run it from inside this folder:
# python compare_with_raw_sdk.py
# Case 3 makes real API calls — needs a valid OPENAI_API_KEY in .env.
import sys
from dataclasses import dataclass

from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import PydanticOutputParser

from chain import build_extraction_chain
from chat_client import create_client, get_structured_reply
from config import load_config
from exceptions import MissingConfigError
from prompts import get_extraction_prompt
from schemas import ExtractedInfo

# how: every input has a name, so neither version returns None here
TEST_INPUTS = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
```

```
    "Ahmed here, please cancel my order",
]

@dataclass
class ComparisonRow:
    input_text: str
    raw_result: ExtractedInfo
    lcel_result: ExtractedInfo
    matched: bool

def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
    # why: copied from the Real-world exercise's Approach 2 — only the
    # chain now comes from chain.py
    client = create_client()
    lcel_chain = build_extraction_chain()
    rows = []
    for text in test_inputs:
        raw_result = get_structured_reply(client, text)
        lcel_result = lcel_chain.invoke({"text": text})
        # why: a name has one right answer; intent wording can differ
        matched = raw_result.name == lcel_result.name
        rows.append(ComparisonRow(text, raw_result, lcel_result, matched))
    return rows

def test_missing_variable() -> None:
    # case 1: a missing {text} fails while building the prompt (Edge cases)
    prompt = get_extraction_prompt()
    try:
        prompt.invoke({})
        print("case 1 (missing text)  -> no error (wrong!)")
    except KeyError:
        print("case 1 (missing text)  -> KeyError")

def test_bad_shape() -> None:
    # case 2: a reply that isn't the shape raises a named error (Failure)
    parser = PydanticOutputParser(pydantic_object=ExtractedInfo)
    try:
        parser.parse("Maria wants a new password")
        print("case 2 (wrong shape)   -> accepted (wrong!)")
    except OutputParserException:
        print("case 2 (wrong shape)   -> OutputParserException")

def test_chain_matches_raw() -> None:
    # case 3: the LCEL chain and Doc04's raw call agree on every name
    rows = run_comparison(TEST_INPUTS)
    matched = 0
    for row in rows:
```

```
        if row.matched:
            matched = matched + 1
    print(f"case 3 (chain vs raw)  -> {matched}/{len(rows)} names match")
    for row in rows:
        if not row.matched:
            print(f"  MISMATCH on: {row.input_text}")
            print(f"    raw:  {row.raw_result}")
            print(f"    lcel: {row.lcel_result}")

def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops here, clearly
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    test_missing_variable()
    test_bad_shape()
    test_chain_matches_raw()

if __name__ == "__main__":
    main()
```

Expected output (case 3 makes real calls):

```
case 1 (missing text)  -> KeyError
case 2 (wrong shape)   -> OutputParserException
case 3 (chain vs raw)  -> 3/3 names match
```

Cases 1 and 2 need no API call — they run for free, every time. If a name ever differs in case 3, the `MISMATCH` lines show both results.

> Back to the Build Task · Hint 1 · Hint 2 · Solution